

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

2026-27
ODD SEMESTER

**INFORMATION ASSURANCE AND SECURITY
(24CS3105)**

ALM – PROTOTYPE REVIEW

submitted by

NAME : B.V.LAXMANA 2420090064

Cluster: **4** Section: **11**

Course Instructor

Dr. Jagan Mohan Reddy Danda

Associate Professor



**KONERU LAKSHMAIAH EDUCATION FOUNDATION, BOWRAMPET
Hyderabad-500043, Telangana, India**

2026-27

Abstract

This project presents a practical study and implementation of fundamental Information Assurance and Security (IAS) concepts through various cryptographic algorithms. The prototype includes Caesar Cipher, Playfair Cipher, Hill Cipher, Vigenère Cipher, Simplified Data Encryption Standard (SDES), Advanced Encryption Standard (AES), Stream Cipher (RC4), Diffie–Hellman Key Exchange, and RSA. These algorithms demonstrate different approaches to protecting information during storage and transmission.

The implementations are developed using Python and follow a client-server communication model, where messages are encrypted before transmission and decrypted at the receiving end. Classical cryptographic algorithms are implemented to understand the basic principles of substitution, key-based encryption, and mathematical transformations. Modern cryptographic techniques such as AES and RC4 demonstrate symmetric encryption, while RSA demonstrates asymmetric encryption using public and private keys. Diffie–Hellman demonstrates the secure establishment of a shared secret key between communicating parties. PyCryptodome is used for the algorithms supported by the library.

The prototype provides hands-on understanding of how cryptographic techniques contribute to confidentiality, integrity, authentication, and secure communication, which are essential objectives of Information Assurance and Security. Through the implementation and testing of these algorithms, the project demonstrates their basic working principles and their application in securing communication between networked systems.

Contents

List of Figures	5
1 Cryptographic Algorithms	1
1.1 Caesar Cipher	1
1.1.1 Algorithm Definition	1
1.1.2 Mathematical Notation	1
1.1.3 Screenshot	2
1.2 Playfair Cipher	2
1.2.1 Algorithm Definition	2
1.2.2 Mathematical Notation	2
1.2.3 Screenshot	3
1.3 Hill Cipher	3
1.3.1 Algorithm Definition	3
1.3.2 Mathematical Notation	4
1.3.3 Screenshot	5
1.4 Vigenère Cipher	5
1.4.1 Algorithm Definition	5
1.4.2 Mathematical Notation	5
1.4.3 Screenshot	6
1.5 Simplified Data Encryption Standard (SDES)	6
1.5.1 Algorithm Definition	6
1.5.2 Mathematical Notation	7
1.5.3 Screenshot	8
1.6 Advanced Encryption Standard (AES)	8
1.6.1 Algorithm Definition	8

1.6.2	Mathematical Notation	9
1.6.3	Screenshots	10
1.7	Stream Cipher (RC4)	11
1.7.1	Algorithm Definition	11
1.7.2	Mathematical Notation.....	12
1.7.3	Screenshot.....	12
1.8	Diffie–Hellman.....	13
1.8.1	Algorithm Definition	13
1.8.2	Mathematical Notation.....	13
1.8.3	Screenshot.....	14
1.9	RSA.....	15
1.9.1	Algorithm Definition	15
1.9.2	Mathematical Notation.....	15
1.9.3	Screenshot.....	16
	GitHub Repository	17

List of Figures

1.1	Caesar Cipher Client-Server Communication	2
1.2	Playfair Cipher Client-Server Communication	3
1.3	Hill Cipher Client-Server Communication	5
1.4	Vigenère Cipher Client-Server Communication	6
1.5	SDES Client-Server Communication	8
1.6	AES File Transfer from Client to Server	10
1.7	AES File Transfer from Server to Client.....	11
1.8	RC4 Client-Server Communication.....	12
1.9	Diffie–Hellman Key Exchange and Communication	14
1.10	RSA Client-Server Communication	16

Chapter 1

Cryptographic Algorithms

1.1 Caesar Cipher

1.1.1 Algorithm Definition

The Caesar Cipher is a classical substitution cipher in which each letter of the plaintext is shifted by a fixed number of positions in the alphabet. The same shift key is used for encryption and decryption.

1.1.2 Mathematical Notation

Let P represent the plaintext character, C represent the ciphertext character, and K represent the shift key.

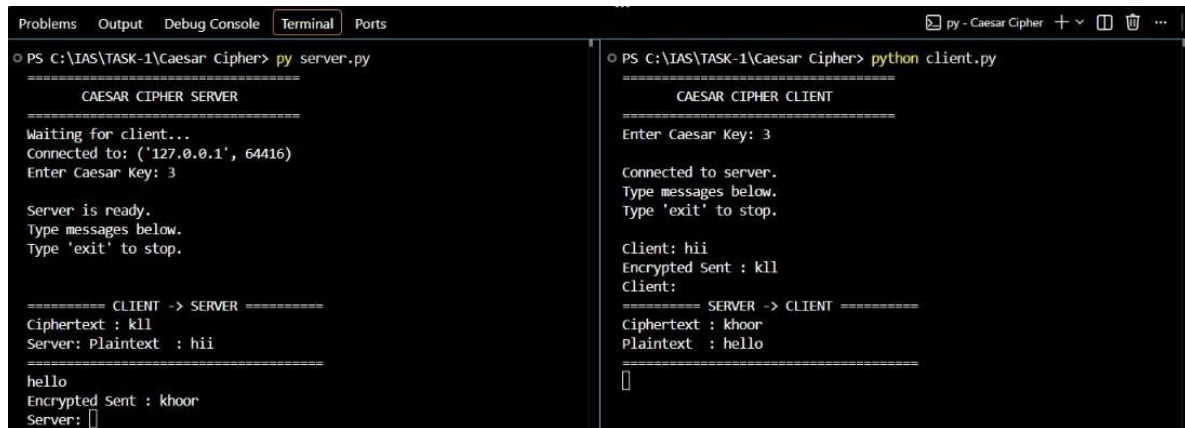
The encryption operation is:

$$C = (P + K) \bmod 26 \quad (1.1)$$

The decryption operation is:

$$P = (C - K) \bmod 26 \quad (1.2)$$

1.1.3 Screenshot



```

Problems Output Debug Console Terminal Ports
PS C:\IAS\TASK-1\Caesar Cipher> py server.py
=====
CAESAR CIPHER SERVER
=====
Waiting for client...
Connected to: ('127.0.0.1', 64416)
Enter Caesar Key: 3

Server is ready.
Type messages below.
Type 'exit' to stop.

===== CLIENT -> SERVER =====
Ciphertext : kll
Server: Plaintext : hii
=====
hello
Encrypted Sent : kloor
Server:

PS C:\IAS\TASK-1\Caesar Cipher> python client.py
=====
CAESAR CIPHER CLIENT
=====
Enter Caesar Key: 3

Connected to server.
Type messages below.
Type 'exit' to stop.

Client: hii
Encrypted Sent : kll
Client:
===== SERVER -> CLIENT =====
Ciphertext : kloor
Plaintext : hello
=====

```

Figure 1.1: Caesar Cipher Client-Server Communication

1.2 Playfair Cipher

1.2.1 Algorithm Definition

The Playfair Cipher is a classical digraph substitution cipher that encrypts two plaintext characters at a time. A 5×5 matrix is generated using a keyword, and character pairs are encrypted according to their positions in the matrix.

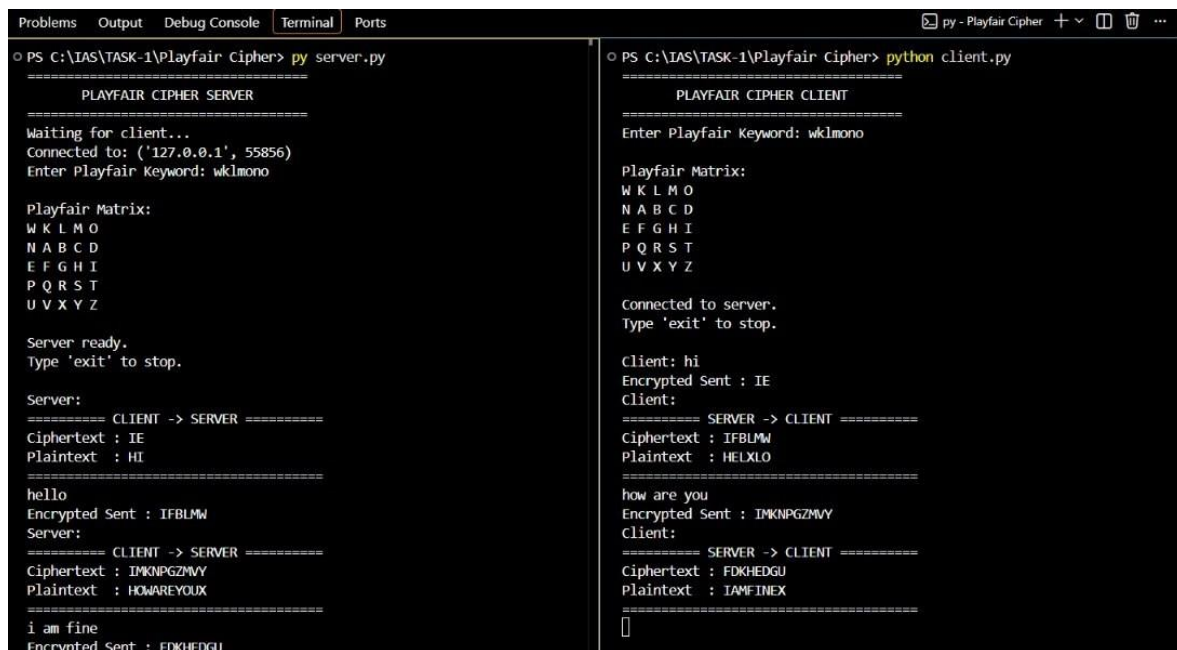
1.2.2 Mathematical Notation

For two characters in the same row, each character is replaced by the character immediately to its right, with wrapping at the end of the row.

For two characters in the same column, each character is replaced by the character immediately below it, with wrapping at the bottom of the column.

For characters forming a rectangle, each character is replaced by the character in the same row and the column occupied by the other character.

1.2.3 Screenshot



```

Problems Output Debug Console Terminal Ports
o PS C:\IAS\TASK-1\Playfair Cipher> py server.py

=====
PLAYFAIR CIPHER SERVER
=====
Waiting for client...
Connected to: ('127.0.0.1', 55856)
Enter Playfair Keyword: wklmono

Playfair Matrix:
W K L M O
N A B C D
E F G H I
P Q R S T
U V X Y Z

Server ready.
Type 'exit' to stop.

Server:
===== CLIENT -> SERVER =====
Ciphertext : IE
Plaintext  : HI
=====

hello
Encrypted Sent : IFBLMW
Server:
===== CLIENT -> SERVER =====
Ciphertext : IMKNPGZMYY
Plaintext  : HOWAREYOUX
=====

i am fine
Encrypted Sent : FDKHEDGU
=====

o PS C:\IAS\TASK-1\Playfair Cipher> python client.py

=====
PLAYFAIR CIPHER CLIENT
=====
Enter Playfair Keyword: wklmono

Playfair Matrix:
W K L M O
N A B C D
E F G H I
P Q R S T
U V X Y Z

Connected to server.
Type 'exit' to stop.

Client: hi
Encrypted Sent : IE
Client:
===== SERVER -> CLIENT =====
Ciphertext : IFBLMW
Plaintext  : HELXLO
=====

how are you
Encrypted Sent : IMKNPGZMYY
Client:
===== SERVER -> CLIENT =====
Ciphertext : FDKHEDGU
Plaintext  : IAMFINEX
=====

```

Figure 1.2: Playfair Cipher Client-Server Communication

1.3 Hill Cipher

1.3.1 Algorithm Definition

The Hill Cipher is a classical polygraphic substitution cipher based on matrix multiplication. A key matrix is used to transform blocks of plaintext characters into ciphertext.

1.3.2 Mathematical Notation

The encryption operation is:

$$C = KP \pmod{26} \tag{1.3}$$

The decryption operation is:

$$P = K^{-1}C \pmod{26} \tag{1.4}$$

where K is the key matrix, K^{-1} is the inverse key matrix, P is the plaintext vector, and C is the ciphertext vector.

For the inverse matrix to exist modulo 26:

$$\gcd(\det(K), 26) = 1 \tag{1.5}$$

1.3.3 Screenshot

```

Problems Output Debug Console Terminal Ports
○ PS C:\IAS\Hill Cipher> py server.py
=====
HILL CIPHER SERVER
=====
Algorithm : Hill Cipher
Key       : [[3, 3], [2, 5]]

Waiting for client...
Connected to: ('127.0.0.1', 57162)

Two-way communication started.
Type 'exit' to stop.

=====
CLIENT -> SERVER
=====

Ciphertext received : TC
Decrypted message   : HI

Enter server message: hello
Encrypted message   : HIOZHN

Waiting for next message...

=====
CLIENT -> SERVER
=====

Ciphertext received : LGOSLCKOZZ
Decrypted message   : HOWAREYOUX

Enter server message: i am fine
Encrypted message   : YQZXLDOT

○ PS C:\IAS\Hill Cipher> python client.py
=====
HILL CIPHER CLIENT
=====
Algorithm : Hill Cipher
Key       : [[3, 3], [2, 5]]

Connected to server.

Two-way communication started.
Type 'exit' to stop.

Enter client message: hi
Encrypted message : TC

=====
SERVER -> CLIENT
=====

Ciphertext received : HIOZHN
Decrypted message   : HELLOX

Enter client message: how are you
Encrypted message : LGOSLCKOZZ

=====
SERVER -> CLIENT
=====

Ciphertext received : YQZXLDOT
Decrypted message   : IAMFINEX

```

Figure 1.3: Hill Cipher Client-Server Communication

1.4 Vigenère Cipher

1.4.1 Algorithm Definition

The Vigenère Cipher is a polyalphabetic substitution cipher that uses a keyword to determine different shifts for successive plaintext characters.

1.4.2 Mathematical Notation

The encryption operation is:

$$C_i = (P_i + K_i) \bmod 26 \quad (1.6)$$

The decryption operation is:

$$P_i = (C_i - K_i) \bmod 26 \quad (1.7)$$

where K_i represents the corresponding key character.

1.4.3 Screenshot

```

PS C:\IAS\Vigenere Cipher> py server.py
=====
VIGENERE CIPHER SERVER
=====
Algorithm : Vigenere Cipher
Host      : 127.0.0.1
Port      : 5000
Key       : SECURITY

Waiting for client connection...
=====
Client connected: ('127.0.0.1', 50871)
Two-way communication started.
Type 'exit' to stop.
=====

CLIENT -> SERVER
Encrypted : zinff
Decrypted : hello

Enter server message: hi client

SERVER -> CLIENT
Plaintext : hi client
Encrypted : zm efzmgr

CLIENT -> SERVER
Encrypted : zinff axpnt
Decrypted : hello server

Enter server message:

PS C:\IAS\Vigenere Cipher> python client.py
=====
VIGENERE CIPHER CLIENT
=====
Algorithm : Vigenere Cipher
Host      : 127.0.0.1
Port      : 5000
Key       : SECURITY

Connecting to server...
Connected to server.

Two-way communication started.
Type 'exit' to stop.
=====

Enter client message: hello

CLIENT -> SERVER
Plaintext : hello
Encrypted : zinff

SERVER -> CLIENT
Encrypted : zm efzmgr
Decrypted : hi client

Enter client message: hello server

CLIENT -> SERVER
Plaintext : hello server
Encrypted : zinff axpnt

```

Figure 1.4: Vigenère Cipher Client-Server Communication

1.5 Simplified Data Encryption Standard (SDES)

1.5.1 Algorithm Definition

Simplified Data Encryption Standard (SDES) is an educational block cipher based on the concepts of permutations, substitutions, key generation, and Feistel-style processing.

SDES demonstrates the basic principles used in block cipher design, including key generation, permutation, substitution, and multiple rounds of encryption.

1.5.2 Mathematical Notation

The basic Feistel operation can be represented as:

$$L_{i+1} = R_i \tag{1.8}$$

$$R_{i+1} = L_i \oplus F(R_i, K_i) \tag{1.9}$$

where L_i and R_i represent the left and right portions of the data block, K_i represents the round key, and F represents the round function.

1.5.3 Screenshot

```

PS C:\IAS\TASK-1\SDES> py server.py
Server
↓
DES Decryption
↓
Original Plaintext File
Purpose:
The purpose of this experiment is to demonstrate encryption, transmission, and decryption of a binary file using DES. The server should recover the same original file after decryption.
This experiment is part of the Active Learning Management (ALM) activity for Cryptography and Network Security.
Client: Sends the encrypted file.
Server: Receives and decrypts the encrypted file.
The plaintext and ciphertext sizes are displayed at both sides to verify the file transfer.
End of File.
----- CIPHERTEXT -----
ccb8da1127aa77b82f694d37818acda61f5d7dac40e8bd3630933648013435c23a625087
3b2e83b7750a73c0102b07e94a5e04a46e0be5fcc5ab040d860fb7d855129357ffb7e412
8269d12e75e6f01a6b34d8a89ce04d49e036416404e6d921959eddf614de2b3b52387e5e
ed56ede8ccfba16def5eca30b72e72923b55e9d6ed299716b568974ebf2ae165b9f2579d
64f15ca553b167723244686dc309da8a8a4ef7481313343163f0afec18db0b43fc04d278
859c11434962bc11c75322a6c1ff196b845cdc1
=====
File decrypted successfully.

PS C:\IAS\TASK-1\SDES> python client.py
SDES / DES FILE TRANSFER CLIENT
=====
Connected to server.
DES Key: 12345678
=====
CLIENT MENU
=====
1. Encrypt and Send File to Server
2. Wait / Receive File from Server
3. Exit
=====
Enter client choice: 1
Enter file path to send: C:\IAS\TASK-1\SDES\client_1MB.pdf
Encrypting file...
=====
CLIENT - FILE TRANSFER
=====
Direction      : CLIENT -> SERVER
File           : client_1MB.pdf
Plaintext size  : 24864 bytes
Ciphertext size : 24880 bytes
=====
----- PLAINTEXT / READABLE CONTENT -----
DES FILE TRANSFER
CLIENT TO SERVER
This file is used for demonstrating secure file transfer using the Data Encryption Standard (DES) algorithm in a Python client-server application.

```

Figure 1.5: SDES Client-Server Communication

1.6 Advanced Encryption Standard (AES)

1.6.1 Algorithm Definition

Advanced Encryption Standard (AES) is a symmetric block cipher used for secure encryption of data. AES operates on 128-bit blocks and supports key sizes of 128, 192, and 256 bits.

The AES prototype uses AES for secure client-server file transfer. The implementation uses the PyCryptodome package.

```
1 from Crypto.Cipher import AES
```

Listing 1.1: PyCryptodome AES Import

1.6.2 Mathematical Notation

The main AES transformations are:

- SubBytes
- ShiftRows
- MixColumns
- AddRoundKey

A standard AES round can be represented as:

$$State' = AddRoundKey (MixColumns (ShiftRows (SubBytes(State)))) \quad (1.10)$$

The final AES round does not perform the MixColumns operation.

1.6.3 Screenshots

```

PS C:\IAS\TASK-2\AES> py server.py

----- PLAINTEXT -----
CLIENT DOCUMENT - DOCX
Information Assurance and Security - ALM-2
Name: P. SATNIKA REDDY
Roll Number: 2420030172
Section: 11

Direction: CLIENT -> SERVER

This Word document is an additional file-type test for the AES file transfer project.
The document is read as binary data, encrypted using AES-256-CBC, transferred through TCP,
and decrypted at the receiving side.

PyCryptodome is used for AES encryption and decryption.

----- CIPHERTEXT -----
8992e1e8d593bed4ab303eb88a79157413238544b1d6b2aAc38cf4ca773a2c6c67b0821dc
84eb7d8d58689768638a317cda2368e7a10cb057e03cb3bfdeecc44791d83539acae10da
e4452b760be40cf4b66bfe44748dccc3245646252e1d3110ef4fd072d75e994659826ca09
d5e1475098d6e7fac4eac081db0c54d0e7d2764e6298bfe2140fca5be89e2a427b52c67
96d265d2ca9364d379944b8647f2e19381cefd426f88c8c062f9011591a1530d82bcb81f
a1fe3368dffeeff2f293ad144eee92a0e8aeb3ea3e9d1b43cdee22a8b837ada6370ea72dd
d69cf08a2c07222e87c24b94542c9838d88ccc26fef18d7f240cb90c16f1a6194f15a35e
c0065605faffdbde09826b091729437aab1a14bdf33362b813cfade22525acfe53f6e22f
b4b198813675bd63fa18ce2ac2a7a653bcab75b1a9cca73ea4efba6bac826817d540ef51
e1363ec69cd5e4caec9eca6488d4a79eb38280f39ffc2bb930735ff837b76f9f0accf9d5
a119482926d4a2b021b5463d8a3e919730e1ad10cf8be11a4d0985878571b13ff1d6dbd2
27a051929e544bd910682690f0408e1a7ce9a84350df54710f88d5d77fca7017cb4dc6b3
8618ad30c212d9a4b69844c4c51db08fe1592d96740f965fd887b47b2b92ee6a0b323233
3f5781601c370591ad9b9c7fccf2bf34f67b02097720e3cc7d84780354b827f4
...[remaining ciphertext not displayed]...

AES decryption completed.
  
```

```

PS C:\IAS\TASK-2\AES> python client.py
AES FILE TRANSFER CLIENT

=====
Connected to server.
AES Algorithm : AES
AES Mode      : ECB
AES Key       : 1234567890123456

Client ready.

=====
CLIENT MENU
=====
1. Encrypt and Send File to Server
2. Wait / Receive File from Server
3. Exit
=====
Enter client choice: 1

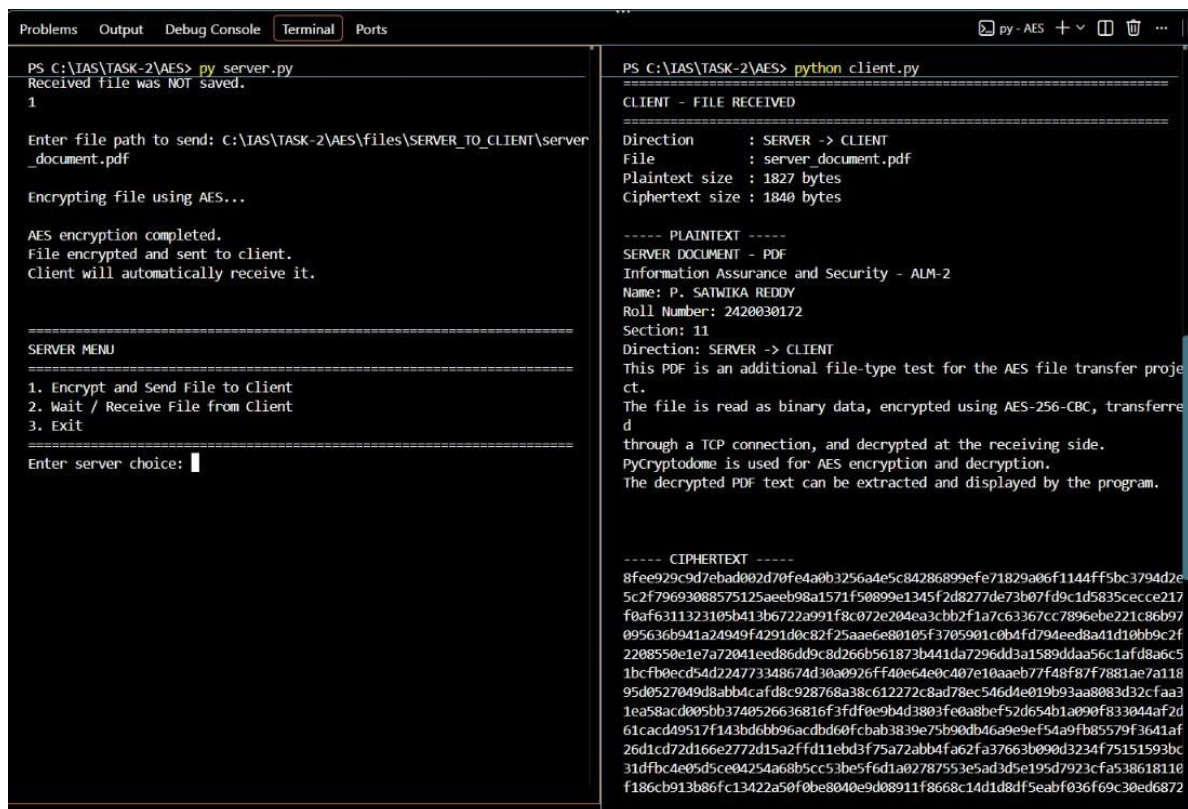
Enter file path to send: C:\IAS\TASK-2\AES\files\CLIENT_TO_SERVER\client_document.docx

Encrypting file using AES...

AES encryption completed.
File encrypted and sent to server.
Server will automatically receive it.

=====
CLIENT MENU
=====
1. Encrypt and Send File to Server
2. Wait / Receive File from Server
3. Exit
=====
Enter client choice:
  
```

Figure 1.6: AES File Transfer from Client to Server



```

PS C:\IAS\TASK-2\AES> py_server.py
Received file was NOT saved.
1

Enter file path to send: C:\IAS\TASK-2\AES\files\SERVER_TO_CLIENT\server_document.pdf

Encrypting file using AES...

AES encryption completed.
File encrypted and sent to client.
Client will automatically receive it.

=====
SERVER MENU
=====
1. Encrypt and Send File to Client
2. Wait / Receive File from Client
3. Exit
=====
Enter server choice: 1

PS C:\IAS\TASK-2\AES> python client.py
=====
CLIENT - FILE RECEIVED
=====
Direction      : SERVER -> CLIENT
File           : server_document.pdf
Plaintext size  : 1827 bytes
Ciphertext size : 1840 bytes

----- PLAINTEXT -----
SERVER DOCUMENT - PDF
Information Assurance and Security - ALM-2
Name: P. SATNIKA REDDY
Roll Number: 2420030172
Section: 11
Direction: SERVER -> CLIENT
This PDF is an additional file-type test for the AES file transfer project.
The file is read as binary data, encrypted using AES-256-CBC, transferred through a TCP connection, and decrypted at the receiving side.
PyCryptodome is used for AES encryption and decryption.
The decrypted PDF text can be extracted and displayed by the program.

----- CIPHERTEXT -----
8fee929c9d7ebad002d70fe4a0b3256a4e5c84286899efe71829a06f1144ff5bc3794d2e
5c2f79693088575125aeeb98a1571f50899a1345f2d8277de73b07fd9c1d5835cecc217
f0af6311323105b413b6722a991f8c072e204ea3cbb2f1a7c63367cc7896ebe221c86b97
095636b941a24949f4291d0c82f25aae6e80105f3705901c0b4fd794eed8a41d10bb9c2f
2208550e1e7a72041eed86dd9c8d266b561873b441da7296dd3a1589ddaa56c1afd8a6c5
1bcfb0ecd54d224773348674d30a0926ff40e64e0c407e10aaeb77f48f87f7881ae7a118
95d0527049d8abb4cafd8c928768a38c612272c8ad78ec546d4e019b93aa8083d32cfaa3
1ea58acd005bb3740526636816f3fdfe09bd3803f0a8bef52d654b1a090f833044af2d
61cacd49517f143bd0bb96acdbd60fcbab3839e75b90db46a9e9ef54a9fb85579f3641af
26d1cd72d166e2772d15a2ffd11ebd3f75a72abb4fa62fa37663b090d3234f75151593bc
31dfbc4e05d5ce04254a68b5cc53be5fd1a02787553e5ad3d5e195d7923cfa538618110
f186cb913b86fc13422a50f0be8040e9d08911f8668c14d1d8df5eabf036f69c30ed6872

```

Figure 1.7: AES File Transfer from Server to Client

1.7 Stream Cipher (RC4)

1.7.1 Algorithm Definition

RC4 (Rivest Cipher 4) is a stream cipher that generates a pseudorandom keystream.

The generated keystream is combined with the plaintext using the XOR operation.

The RC4 prototype uses the ARC4 implementation provided by PyCryptodome.

```
1 from Crypto.Cipher import ARC4
```

Listing 1.2: PyCryptodome RC4 Import

1.7.2 Mathematical Notation

The encryption operation is:

$$C_i = P_i \oplus K_i \quad (1.11)$$

The decryption operation is:

$$P_i = C_i \oplus K_i \quad (1.12)$$

where K_i represents the generated keystream.

The RC4 key scheduling algorithm generates the initial state, and the pseudo-random generation algorithm produces the keystream.

1.7.3 Screenshot

```

Problems Output Debug Console Terminal Ports
o PS C:\IAS\Stream Cipher (RC4)> py server.py

=====
RC4 STREAM CIPHER SERVER
=====

Algorithm : RC4
Package   : PyCryptodome
Key       : SECRETKEY

Waiting for client...
Connected to: ('127.0.0.1', 50061)

Two-way communication started.
Type 'exit' to stop.

=====
CLIENT -> SERVER
=====

Ciphertext:
50e2cf

Decrypted plaintext:
hi

Enter server message: hello

Encrypted ciphertext:
50ee83f2db

Waiting for next message...

o PS C:\IAS\Stream Cipher (RC4)> python client.py

=====
RC4 STREAM CIPHER CLIENT
=====

Algorithm : RC4
Package   : PyCryptodome
Key       : SECRETKEY

Connected to server.

Two-way communication started.
Type 'exit' to stop.

Enter client message: hi

Plaintext:
hi

Encrypted ciphertext:
50e2cf

=====
SERVER -> CLIENT
=====

Ciphertext received:
50ee83f2db

Decrypted plaintext:
hello

```

Figure 1.8: RC4 Client-Server Communication

1.8 Diffie–Hellman

1.8.1 Algorithm Definition

Diffie–Hellman is a key exchange algorithm that allows two parties to establish a shared secret over a communication channel without directly transmitting the secret key.

Both parties use common public parameters and their own private values to calculate the same shared secret.

1.8.2 Mathematical Notation

Let P be a public prime number and G be a public generator.

The client selects private value a and calculates:

$$A = G^a \bmod P \quad (1.13)$$

The server selects private value b and calculates:

$$B = G^b \bmod P \quad (1.14)$$

The client calculates the shared secret:

$$S = B^a \bmod P \quad (1.15)$$

The server calculates the shared secret:

$$S = A^b \bmod P \quad (1.16)$$

Therefore:

$$B^a \bmod P = A^b \bmod P \quad (1.17)$$

and both parties obtain the same shared secret.

1.8.3 Screenshot

```

Problems Output Debug Console Terminal Ports
o PS C:\IAS\Diffie-Hellman> py server.py
=====
DIFFIE-HELLMAN SERVER
=====
Algorithm : Diffie-Hellman Key Exchange
Public P : 23
Public G : 5
Private Key: 6

Waiting for client...
Connected to: ('127.0.0.1', 59985)

Client Public Key : 19
Server Public Key : 8

Shared Secret Key : 2

=====
Diffie-Hellman key exchange completed.
Two-way communication started.
Type 'exit' to stop.
=====

CLIENT -> SERVER
Message : hi server
Enter server message: hello server
o PS C:\IAS\Diffie-Hellman> python client.py
=====
DIFFIE-HELLMAN CLIENT
=====
Algorithm : Diffie-Hellman Key Exchange
Public P : 23
Public G : 5
Private Key: 15

Connected to server.

Client Public Key : 19
Server Public Key : 8

Shared Secret Key : 2

=====
Diffie-Hellman key exchange completed.
Two-way communication started.
Type 'exit' to stop.
=====

Enter client message: hi server

SERVER -> CLIENT
Message : hello server

Enter client message:

```

Figure 1.9: Diffie–Hellman Key Exchange and Communication

1.9 RSA

1.9.1 Algorithm Definition

RSA is an asymmetric cryptographic algorithm that uses a public key and a private key. The public key can be used for encryption, while the corresponding private key is used for decryption.

The RSA prototype uses the RSA functionality provided by PyCryptodome.

```
1 from Crypto.PublicKey import RSA
2 from Crypto.Cipher import PKCS1_OAEP
```

Listing 1.3: PyCryptodome RSA Imports

1.9.2 Mathematical Notation

Two prime numbers p and q are selected.

The modulus is calculated as:

$$n = pq \tag{1.18}$$

Euler's totient function is:

$$\phi(n) = (p - 1)(q - 1) \tag{1.19}$$

The public key is represented by (e, n) and the private key by (d, n) .

The encryption operation is:

$$C = M^e \bmod n \quad (1.20)$$

The decryption operation is:

$$M = C^d \bmod n \quad (1.21)$$

where M is the plaintext message and C is the ciphertext.

1.9.3 Screenshot

```

Problems Output Debug Console Terminal Ports
PS C:\IAS\RSA> py server.py
Connected to: ('127.0.0.1', 62056)

Client public key received.
Server public key sent.

=====

RSA key exchange completed.
Two-way encrypted communication started.
Type 'exit' to stop.

=====

CLIENT -> SERVER

=====

Encrypted ciphertext:
82104533f71c3e0290998bd28372de6807f282174c851280e7be63602c7b51efc385a1d4
b9c973166c2c8443b05bce03dddf86db538c8170eb25b20b584495ef410ef5c20ad465
563e31312eddc4f463338b24b56b10dcfca9544f1e69b90857fb77cf2cf00e4ce486d456
20c0fc782b190c43b7630b0fa8ca345e80c7cd336a177968347c94fe715663f6b3d8e7c1
4116c8e99f0826f27d837a56354339e3a6926168c4d49379a7145d9bab7c96e58b7cde14
d459375360cda547dbbbe4b48ed7d8c54828eb23358428dd86852613654b98d203e413a
6429d2fa2c966130042262557d175def0b6a5d5dc7ae44c4410456e637d1600818ac744e
bef2f0d3

Decrypted plaintext:
hello

Enter server message: hi client

Encrypted ciphertext:
55966041e3289cd78d56f5e1bb259847dc2059026bd1eddefff507e87cc98383e423da16
6a2191f761d8c2cda7ab9c02e8af7a47b2bdb2f22b2a2d8cb63c872292cab67b89f0830f
5089508ad507315dd0178556854ac5a1197dcb598808136288b8336e4a1f80c72a98dda9
c5a18c27d64ed821862b9e7420b4ae10881e8d61d5c56014ea5f74b16beccb44ad615707
1a4f48034531c97a8cce1730b99b281cd6ba5646624789cee7c00ea8ecdf798c52bf65c6
918807a64960619f4a99d373377d19b1ba3b9fe219ef5215645f3b75c60825843811dc01
042d548dd78cb847e403ac848e46a2bbfcef8ba0237c94df84263fbc6aa3313e7929f68d

PS C:\IAS\RSA> python client.py
Server public key received.
Client public key sent.

=====

RSA key exchange completed.
Two-way encrypted communication started.
Type 'exit' to stop.

=====

Enter client message: hello

Plaintext:
hello

Encrypted ciphertext:
82104533f71c3e0290998bd28372de6807f282174c851280e7be63602c7b51efc385a1d4
b9c973166c2c8443b05bce03dddf86db538c8170eb25b20b584495ef410ef5c20ad465
563e31312eddc4f463338b24b56b10dcfca9544f1e69b90857fb77cf2cf00e4ce486d456
20c0fc782b190c43b7630b0fa8ca345e80c7cd336a177968347c94fe715663f6b3d8e7c1
4116c8e99f0826f27d837a56354339e3a6926168c4d49379a7145d9bab7c96e58b7cde14
d459375360cda547dbbbe4b48ed7d8c54828eb23358428dd86852613654b98d203e413a
6429d2fa2c966130042262557d175def0b6a5d5dc7ae44c4410456e637d1600818ac744e
bef2f0d3

SERVER -> CLIENT

=====

Ciphertext received:
55966041e3289cd78d56f5e1bb259847dc2059026bd1eddefff507e87cc98383e423da16
6a2191f761d8c2cda7ab9c02e8af7a47b2bdb2f22b2a2d8cb63c872292cab67b89f0830f
5089508ad507315dd0178556854ac5a1197dcb598808136288b8336e4a1f80c72a98dda9
c5a18c27d64ed821862b9e7420b4ae10881e8d61d5c56014ea5f74b16beccb44ad615707
1a4f48034531c97a8cce1730b99b281cd6ba5646624789cee7c00ea8ecdf798c52bf65c6
918807a64960619f4a99d373377d19b1ba3b9fe219ef5215645f3b75c60825843811dc01
042d548dd78cb847e403ac848e46a2bbfcef8ba0237c94df84263fbc6aa3313e7929f68d
6a56bf12

```

Figure 1.10: RSA Client-Server Communication

GitHub Repository

The complete implementation of the Information Assurance and Security prototype is available in the following GitHub repository:

<https://github.com/satwika06bhaskar-design/IAS-2420030172.git>